Core after the tensor extraction: release, semantic reads, debt and cell storage

Created-at: 2026-09-30 20:34 +02:00 Status: **PlanOnly**

Request. On 2026-09-30 the user said: "Foglalkozzunk most a core-al" (let us deal with the Core now). Asked which direction, they chose all four: release, `ir-semantic-reads`, technical debt and `cell-storage-views`. This plan re-baselines each against Core `feat/tensor-extraction` `b1bc1eb` and asks for the decisions each needs. It edits no protected item. `hymeko_core` is `lockdown: full` (`CORE.YAML`), so every Core source change below needs its own `APPROVED-CORE-EDIT` slug.

Evidence method. Measurements were run on this machine: tests, clippy and rustfmt. Three read-only audits supplied file-and-line evidence, summarised here: the semantic reads re-baseline, the consumers of the resolution `Index`, and the cell storage re-evaluation. Nagare runs on another machine and is out of scope.

1 Baseline (measured 2026-09-30)

- Core `feat/tensor-extraction` `b1bc1eb`:
  - It merges the 09-23..25 work (`9106368`) with `main` `c47ea64`.
  - `cargo test --workspace`: 696 passed, 0 failed, 20 ignored. `cargo clippy --workspace --all-targets`: clean.
  - rustfmt reports 30 findings; `main` has 47.
  - PR #14 is open. Every other Core branch on `origin` is already in `main`.
- Tensor `feat/tensor-extraction` `091f4f1`:
  - It is two commits over `main` `87116b6`: the numeric owner extraction and the arena.
  - `cargo test --workspace --all-features`: 327 passed, 0 failed, 1 ignored. It has no PR yet.
- Readers pin these two revisions on their own `feat/*` branches: graph, transformation (PR #4), Kawamirai, hive, xr, `hymeko_web`, network, transformation-formula and core_cpp.
- `CORE.YAML` `merkle-hash-removal` records a wire consequence. Readers older than the change reject new payloads, because their `IrWire` requires `decl_hash`/`arc_hash`. HIVE deployments must therefore move in the same wave.

2 R: release

R1 **Tensor PR**, `feat/tensor-extraction` → `main`. Anonymous Gitea API access sees no private repository, so the agent can open it only over SSH with Gitea's AGit flow (`git push origin HEAD:refs/for/main -o title=.. -o description=..`). The alternative is the user's web link.

R2 **Merge order:** tensor first, then Core #14. Both without squash, so every pinned commit (`091f4f1`, `b1bc1eb`) stays reachable from `main`. Then no reader has to repin for the release; repinning to `main` merge commits can happen at each reader's next change.

R3 **Reader PRs, provider before consumer:**

(a) graph, then transformation #4;

(b) Kawamirai;
(c) hive (`feat/tensor-extraction`, then `feat/pose-sample-owner`);
(d) xr;
(e) `hymeko_web`, with its stack `feat/tensor-extraction` → `pose-sample-owner` → `browser-suite-fixes` → `check-gate`;
(f) network, transformation-formula and core_cpp, which are independent of each other.

R4 **Deployment:** HIVE on the Pi first, because it is the wire provider for every client. Then republish the live room from middleware `8ee914e`. The Quest follows once the simulator reads scene-state v6 (Kawamirai plan, J3).

Decisions.

- DR1: who merges; the agent only on explicit request.
- DR2: no squash.
- DR3: when HIVE is deployed.
- DR4: whether the agent opens PRs through AGit.

3 S: semantic reads (re-baseline of `2026-09-17-ir-semantic-reads`)

None of that plan's Core API was built, and nothing in `hymeko_core` walks bases today. Meanwhile `hymeko_query` (transformation) became the practical owner:

- `effective_child`/`effective_value` (`fields.rs`) and `Predicate::InheritsFromId`;
- Geometry and `hymeko_webxr` read effective fields through it.

That contradicts the rejected-alternatives section of the 09-17 plan ("Query is not the semantic owner") and `AGENTS.md`, which assigns borrowed semantic queries to Core.

Gap	State now
G1 base API, is-a by DeclId	partial, in `hymeko_query` (depth 8, no cycle error, no policy)
G2 effective fields	partial, in `hymeko_query`; Geometry and webxr use it
G3 multi-base order	open: DFS in authored order; the diamond case (A1) returns R, not B
G4 provider identity / own slot	partial: provider DeclId only; webxr's `own` is private
G5 structural updates	open: value updates are leaf-only
G8 which bases count	open, and split: graph and network follow `<isa>` only, query and webxr all bases
G9 const hazards, G10 emitters drop base sign/tags, G11 ecosystem units	open
C3, C4 private inherited-ref walks	closed in transformation; C4 remains in the formula worktree

About eleven private base walks remain outside the query crate:

- graph `meta_resolve` and network `model`;
- dynamics and optimize `read`, and geometry `kinematic`;
- webxr `lib` and `query`;
- `hymeko_wasm` `summary`/`document`/`inspector`;
- the C port `hy_pattern.c`.

Reduced scope.

- S1 Core (additive): a borrowed `bases_of(DeclId)` and `is_a(decl, base, BasePolicy)` with a typed cycle error. `BasePolicy` is `All` or `Tagged(isa)`.
- S2 `hymeko_query`:

  - `effective_child` reports its provider and whether the value is local;
  - webxr's `own` rule becomes a public slot check for writers (HIVE, robotics).
- S3: consumers drop their private walks.
- S4: a diamond fixture that fixes DFS vs C3 before any consumer relies on it.

The full `BaseView` with cached linearisations, C3/C4 in transformation and S6 as written are obsolete.

Decisions.

- DS0: owner of borrowed semantic queries, Core or `hymeko_query`. It must be settled first; the code and `AGENTS.md` disagree.
- DS1: `All` or `Tagged(isa)`. The code is already split.
- DS2: ratify DFS, or reopen C3.
- DS3: the approval slug for S1 (`ir-semantic-reads`, reduced).
- DS4: whether G9, G10 and G11 become separate tasks.

4 T: technical debt

T1 rustfmt. There are 30 findings on the branch and 47 on `main`.

- Lockdown `full` covers formatting, so this needs a slug, e.g. `core-rustfmt`.
- It is one formatting-only commit, with tests and clippy re-run after it.
- No other Core branch is open, so there is nothing to conflict with.

T2 the resolution `Index` in compiled programs and snapshots. The `CompiledProgram.idx` comment says only test consumers remain. That is not accurate:

- Production reads: Core 6 (encode, decode validation, value updates, symbol compaction); `hymeko_web` 2 (`source_edit/references.rs` resolves through `module_aliases`, and `document.rs` rebuilds a program); hive 1 (`http/live.rs` re-encode).
- Test and tooling reads: about 25 in Core, 1 in Kawamirai, 1 in xr and 3 in the core_cpp oracle.
- The C runtime reads `index` and `by_path` from the wire, and exposes aliases publicly.

What `build_path_index` can and cannot replace:

- It replaces every `by_path` use. Decode already proves the wire copy equals the derived one.
- It cannot replace `aliases`: the root `using` aliases have no home in the IR.
- It cannot replace per-module alias environments: they are crate-private, not serialised, and cleared on decode. So a program installed from a snapshot already has a weaker `idx` than a fresh compile.

Why the wire cannot simply drop the field:

- Schema 1 requires `index`, in Rust and in C, and the C decoder rejects unknown fields.
- HIVE archives store v1 documents.

Sequence, providers before consumers:

- T2a Core, additive:
  - a home for aliases in the IR;
  - a Core resolver for source references;
  - `encode` derives `by_path` itself;
  - validators that check the IR alone;
  - deprecations;
  - one `DerivedPathIndex` constructor (xr and middleware still call the WIP-only `DerivedPathIndex::new`);
  - removal of dead `Index` uses in `pretty_print` and in benches/examples.
- T2b readers: accept an optional `index` and a schema v2, Rust and C; keep v1 decoding for the archives.
- T2c consumers migrate: web, hive, tests and oracle.

- T2d producers emit v2, but only after HIVE, Quest, simulator, middleware and firmware ship T2b.
- T2e breaking release: remove `idx`, `CborPayload.index` and `index()`; `Index` becomes crate-private.

Decisions.

- DT1: slug for T1.
- DT2: where aliases live in the IR.
- DT3: schema v2 timing, tied to the HIVE deployment.
- DT4: slugs for T2a and T2e.

5 C: cell storage (re-evaluation of `2026-09-21-cell-storage-views`)

Still unimplemented. The planned first slice, `CellStorage { ir }` inside `PreparedIr`, still compiles against the unchanged `PreparedIr`. But it is the "cosmetic wrapper" the plan itself warns against: it adds no invariant, removes no clone, and its name clashes with Graph's `CellTopology`.

Since 09-21 the baseline has moved:

- hash fields are gone and `bind_decl` is checked;
- `IrConstruction` is the crate-private append-only writer; `ir_tables!` exists;
- `CompiledProgram` carries `CanonicalIdentity`, and `CompiledModel` a semantic cache;
- the compute arena moved to tensor (a buffer pool, not an IR handle);
- Graph gained `CellTopology`;
- Nagare holds `PreparedIr` and clones the IR to build one.

The real duplication today is two owners of one IR, `CompiledProgram.ir` and `PreparedIr.ir`. Because of it, Nagare clones the IR and `hymeko_web` builds its own incidence index.

Smallest valuable slice (C1).

- `CompiledModel` derives its incidence index once and lends it: `OnceLock<IncidenceIndexData>`, with `incidences() -> Result<IncidenceView, IncidenceError>`. It follows the `semantic` cache precedent and stays out of the wire.
- Tests: pointer identity, error parity, zero allocation after first use.
- Size: about 0.5–1 day in Core (2–3 files). Adoption in `hymeko_web` is separate, about half a day.

Phases 2–8 need re-planning against Canon, `CanonicalIdentity`, symbol compaction and `CellTopology` before any estimate.

Decisions.

- DC1: a derived cache on the model (C1), or `PreparedIr` as the program's IR owner (breaks `pub ir` for web, XR, hive and robotics).
- DC2: who owns the "cell" vocabulary, Core records or Graph's `CellTopology`.
- DC3: whether to seal the public `Ir` fields behind `IrConstruction`.
- DC4: whether `MatroidRef` gets an `IncidenceView` adapter.
- DC5: the approval slug (e.g. `model-incidence-cache`).

6 Order, verification, rollback

1. R needs no Core source edit and can start as soon as DR1–DR4 are decided.
2. The additive Core items follow independently, each on its own branch after its slug: T1, C1, S1, T2a. T1 goes first, so the others are formatted from the start.
3. The consumer and wire steps follow: S2, S3, T2b–T2e.

Every Core item is verified the same way:

- `cargo test --workspace`, clippy and rustfmt;
- `cargo tree -d` in each reader it touches, and that reader's own tests;
- the `hymeko_web` gate (`scripts/check.mjs`) for web-facing changes;
- the core_cpp oracle for anything on the wire.

Every step is one branch, so its rollback is reverting that branch. No step changes digest bytes, and only T2d/T2e change the wire.

